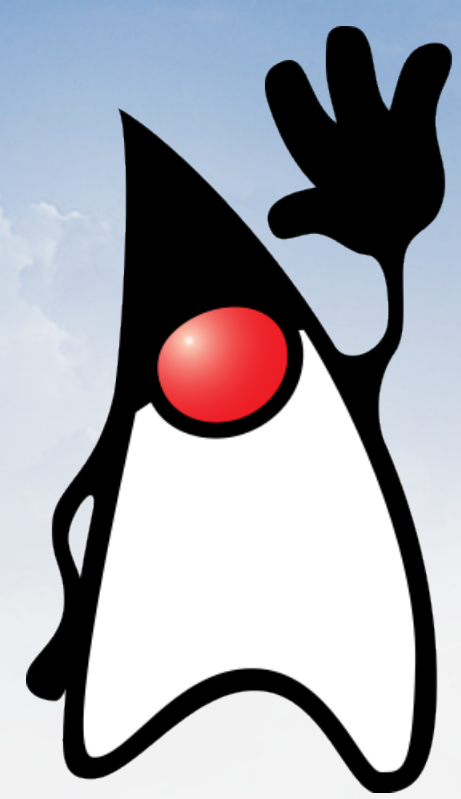




Java Strings

COP 3330
Department of Computer Science
University of Central Florida
Dr. Bacanli

What do we know about Strings?

- A string is an array of characters.
- Each element in the array is some sort of character.

H	a	r	r	y	P	o	t	t	e	r
0	1	2	3	4	5	6	7	8	9	10

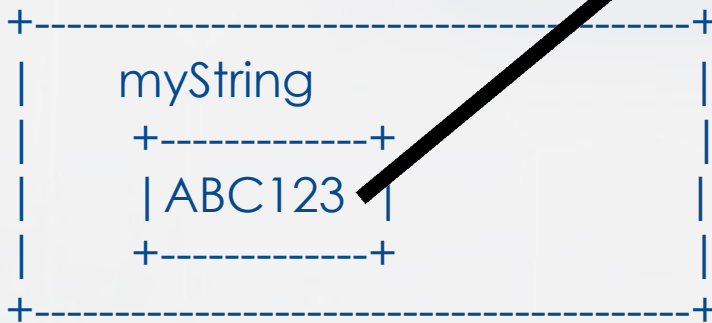
Java and Strings

- Strings are a built in class type.
 - NOT PRIMITIVE!
- They can contain any numeric and alphanumeric values
- Always initialize Strings to something
 - `String name = "";`
- In Java, strings know their size. No need to calculate every time.
- Since Strings are char arrays, the indexing start from 0.

```
public class Sample
{
    public static void main(String [] args)
    {
        String myString = "HarryPotter";
    }
}
```

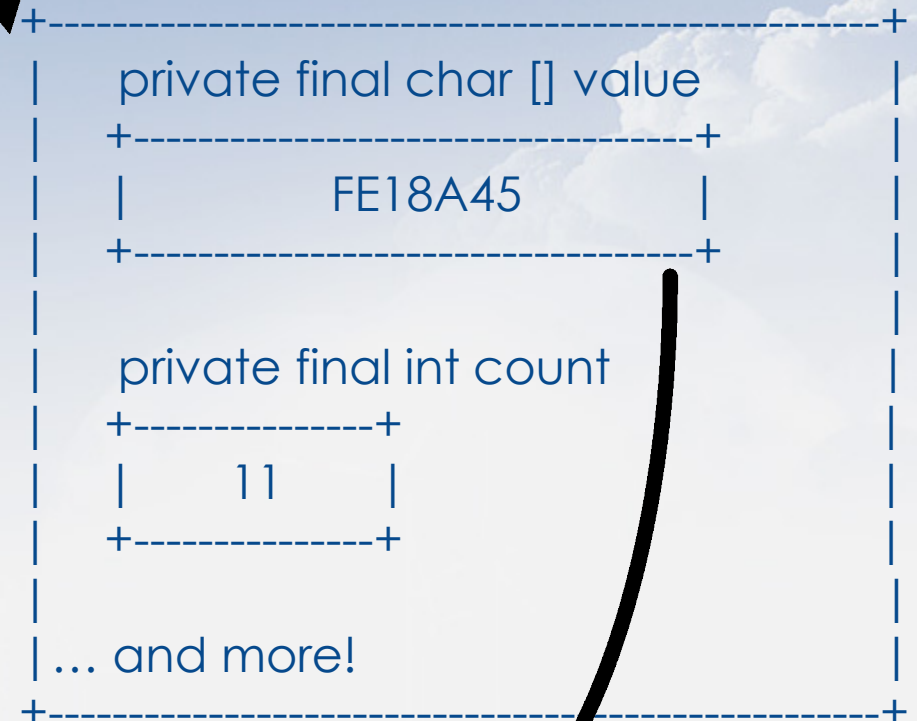
Stack Space

In main()

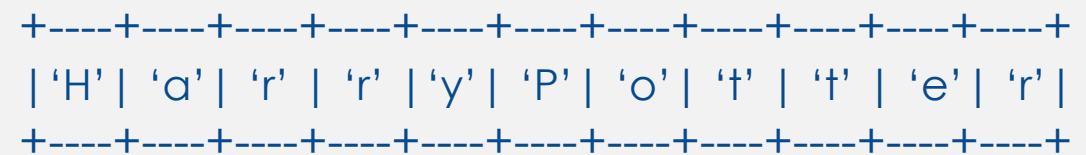


Heap Space

ABC123



FE18A45



What do you think happens in memory when the following three lines are executed?

```
String name;
```

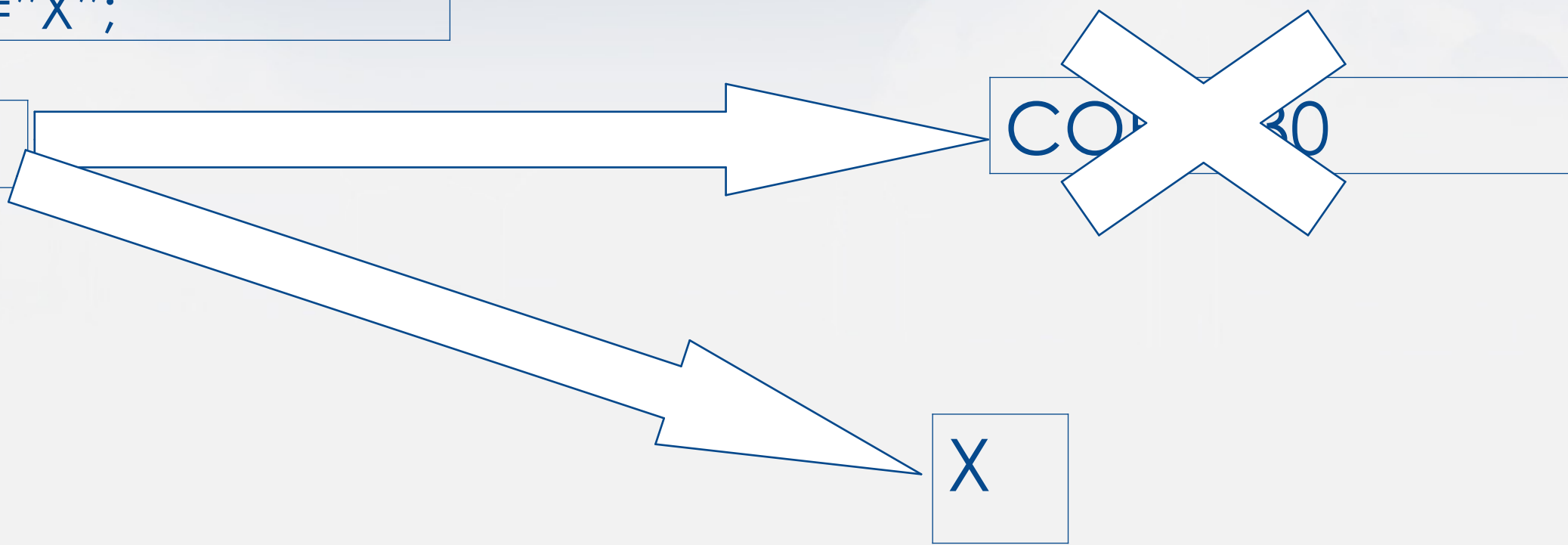
```
name="COP3330";
```

```
name="X";
```

name

COP3330

X



Immutable vs Mutable Objects

- Based on the pass example, you have witnessed that Strings are immutable.
 - Immutable means you cannot change anything about the object!
 - Even if you were try and update a character index, Java will not compile!
- Mutable objects are the opposite of immutable

String initialization

There are some escape sequences that you can use:

`\n` : newline
`\t` : tab
`\\` : single slash
`\\\\` : double slash
`\"` : Double Quote

```
String name = "Test\nthis\nString";
```

```
String name = "Test\\Nontest options";
```

```
String name = "Testing \"is\" testing ";
```

backslash means operate
the next char differently

```
System.out.println(name);
```

```
System.out.println(name);
```

```
System.out.println(name);
```

length()

- A get method that retrieves the length attribute of a String object.
- Runs in $O(1)$ as it is only a lookup method.
 - The size of string is written on somewhere
 - It is being checked whenever we need the length of string
- Java Strings are char arrays, does not end with Null character!

string is test101
This length is 12

```
package stringTest;

public class StringTest{
    public static void main(String[] args){
        String name = "test101\nThis";
        System.out.println("string is "+name +" length is " +name.length());
    }
}
```

equals

```
String name = "COP3330";  
String anotherName = "COP3330";  
System.out.println(name == name);  
System.out.println(name.equals(anotherName));
```

If you want to compare the contents of strings, use equals method!

String Concatenation

- We can use the + operator to append a String to another String.
 - Something to think about?
 - Are there runtime restrictions?

```
package stringTest;

public class StringTest{
    public static void main(String[] args){
        String name = "java";
        name = name + " is " + name; // name reference is changed
        System.out.println(name); // java is java
    }
}
```



String Tokenizer

```
package splittest;
import java.util.StringTokenizer;
public class splitTest{
    public static void main(String[] args){
        String sentence = "This.is.an.example.sentence";
        StringTokenizer st= new StringTokenizer(sentence, ".");
        while(st.hasMoreTokens()){
            String text=st.nextToken();
            // do NOT call nextToken again!
            System.out.print(text+" ");
        }//end of while
    }//end of main
} //end of class
```

This is an example sentence



split

```
package splittest;
// No need to import anything
public class splitTest{
    public static void main(String[] args){
        String sentence= "ThisXisXanXexampleXsentence";
        String[] arr= sentence.split("X");
        System.out.println(arr[0]); //first token
        System.out.println(arr[arr.length-1]); // last token
    }
}
```

This
sentence

charAt()

- Returns the char value at the specified index
- No negative index in strings! $0 \leq \text{index} < \text{length}()$
- What happens if the index is out of bounds?
 - For the below code, change the upper and lower limits of *i*
 - See what happens.

```
String name= "Test";  
//Traversing the string, character by character  
for(int i=0;i<name.length();i++){  
    char myChar = name.charAt(i);  
    System.out.print(myChar);  
}
```

compareTo()

- Compares two Strings
- Returns the value 0 if the argument string is equal to this string; a value less than 0 if this string is lexicographically less than the string argument; and a value greater than 0 if this string is lexicographically greater than the string argument.

```
package stringTest;
```

```
public class CompareToExample{  
    public static void main(String args[]){  
        String s1="hello";  
        String s2="hello";  
        String s3="meklo";  
        String s4="hemlo";  
        String s5="flag";  
  
        System.out.println(s1.compareTo(s2));  
        //0 because both are equal  
  
        System.out.println(s1.compareTo(s3));  
        //-5 because "h" is 5 times lower than "m"  
  
        System.out.println(s1.compareTo(s4));  
        //-1 because "l" is 1 times lower than "m"  
  
        System.out.println(s1.compareTo(s5));  
        //2 because "h" is 2 times greater than "f"  
    } //end of main  
} //end of class
```

Generating Substrings in Java

- String class has a method called substring(x,y)
 - Two Parameters
 - x is the first position of original substring (inclusive)
 - y is the last position of original substring to go up to (exclusive)
- Example
 - String str1 = "Java is Fun!";
 - String str2 = str1.substring(0,3); // [0,3)
 - What will str2 store?

Can we give negative index number to substring method?

No

concat()

- Concatenates a string to another string
- <https://stackoverflow.com/questions/47605/string-concatenation-concat-vs-operator>
- Prefer using +
- Concat returns a new string, doesn't modify the parameters

```
String firstName = "John ";  
String lastName = "Doe";  
System.out.println(firstName.concat(lastName));
```

int indexOf(), boolean contains()

```
class HelloWorld {  
    public static void main(String[] args) {  
        String name = "This is a string";  
        int pos=name.indexOf("g");  
        int pos1=name.indexOf("is");  
        int pos2=name.lastIndexOf("is");  
        boolean containsBoolean=name.contains("tri");  
        System.out.println("position of g "+ pos);  
        System.out.println("first position of is "+ pos1);  
        System.out.println("last position of is "+ pos2);  
        System.out.println("does name contain \"tri\"? "  
+containsBoolean);  
    }  
}
```

position of g 15
first position of is 2
last position of is 5
does name contain "tri" true

What if the string does
not exist in the main
string?

indexOf(..) will return -1.
contains() will return false.

Other String Methods

- There are countless methods for String Class.
- Java Docs is a great resource to look up each of the methods as it is maintained by the creators of Java.
- <https://docs.oracle.com/en/java/javase/11/docs/api/java.base/java/lang/String.html>

<code>char charAt(int index)</code>	Returns the char at the specified location.
<code>int compareTo(String other)</code>	Returns negative value if the string comes before other in dictionary order, a positive value if the string comes after other in dictionary order, or 0 if the strings are equal
<code>boolean isEmpty()</code>	returns true if the string is empty
<code>boolean startsWith(String prefix)</code> <code>boolean endsWith(String suffix)</code>	returns true if the string starts with prefix or ends with suffix
<code>int indexOf(String str)</code> <code>int indexOf(String str, int fromIndex)</code>	returns start of the first substring equal to the string str, starting at index 0 or at fromIndex, or -1 if str does not occur in this string.
<code>String toLowerCase()</code> <code>String toUpperCase()</code>	returns a new string containing all characters in the original string, with uppercase characters converted to lowercase, or lowercase characters converted to uppercase

Check out

<https://docs.oracle.com/en/java/javase/11/docs/api/java.base/java/lang/String.html> for more methods in Strings

The StringBuilder Class

- Another String class where the objects are mutable.
 - append(String param)
 - Can insert character(s)
 - length()
 - Can look up number of characters in the string.
 - toString()
 - return the String value
- No need to import anything to use StringBuilder

4 Answers

Sorted by: Highest score (default)



I think the use of `StringBuilder` vs `+` really depends on the context you are using it in.

119

Generally using JDK 1.6 and above the compiler will automatically join strings together using `StringBuilder`.



```
String one = "abc";  
String two = "xyz";  
String three = one + two;
```



This will compile `String three` as:

```
String three = new StringBuilder().append(one).append(two).toString();
```

```
StringBuilder sb= new StringBuilder("");  
for(int i=0; i<5; i++){  
    sb.append(i+ " ");  
}
```

```
String str1= "";  
for(int i=0; i<5; i++){  
    str1=str1+i+ " ";  
}
```

If you are making concatenation
not inside a loop use +

If you are using a for loop,
StringBuilder is better.

StringBuilder Functions

```
package HelloWorld;
class HelloWorld {
    public static void main(String[] args) {
        StringBuilder sb=new StringBuilder("TEST");
        System.out.println(sb.toString());

        int number= sb.indexOf("EST");
        System.out.println("Position of EST " + number);
        System.out.println("TEST equals TEST? "+ sb.toString().equals("TEST") );
    }
}
```

TEST

Position of EST 1

TEST equals TEST? true

little exercise

```
/*  
This function counts all occurrences of a needle string in hay string  
@param hay as the string to be searched  
@param needle as the string to be found  
@return number of occurrences  
*/  
public static int allOccurrences(String hay,String needle){  
    //use only indexOf and substring methods  
}  
  
public static void main(String[] args) {  
    System.out.println(2==allOccurrences("mississippi", "ss")); //returns true  
    System.out.println(3==allOccurrences("tesssamenssa", "ss")); //returns true  
    System.out.println(0==allOccurrences("northdakota","eel")); //returns true  
}
```